

Amrita Vishwa Vidyapeetham - School of Computing, Chennai

Topic: Graph (10 Challenge Programs) | Roll No: CH.SC.U4CSE25262

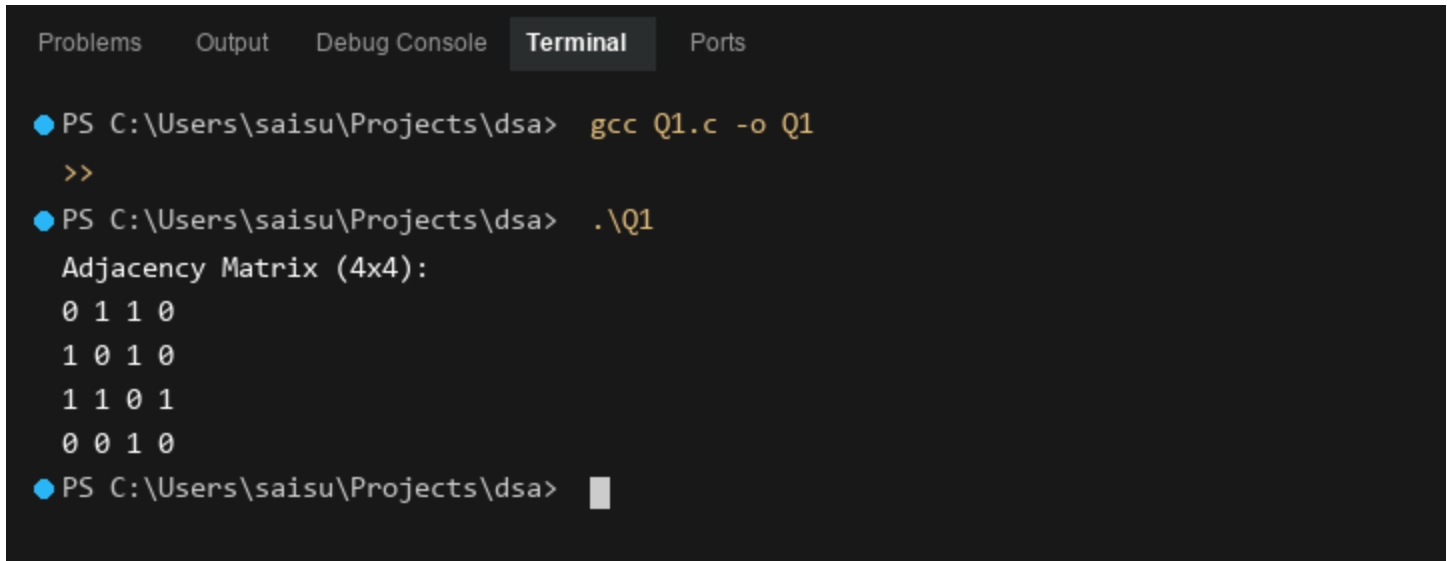
Question 1: Graph Representation via Adjacency Matrix and List

Problem Description:

Represent a directed/undirected graph with V vertices and E edges using both 2D Adjacency Matrix and dynamic Adjacency Lists.

Sample Input: $V=4$, $E=4$: (0,1), (0,2), (1,2), (2,3) | Sample Output: Adjacency Matrix Printed

Output Screenshot:



```
Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\saisu\Projects\dsa> gcc Q1.c -o Q1
>>
PS C:\Users\saisu\Projects\dsa> .\Q1
Adjacency Matrix (4x4):
0 1 1 0
1 0 1 0
1 1 0 1
0 0 1 0
PS C:\Users\saisu\Projects\dsa> 
```

Question 2: Breadth First Search (BFS) Traversal

Problem Description:

Implement Breadth-First Search (BFS) graph traversal starting from a source vertex using a FIFO queue and a boolean visited array.

Sample Input: Start vertex: 2 | Sample Output: BFS Traversal: 2 0 3 1

Output Screenshot:



The screenshot shows a terminal window with the following content:

```
Problems  Output  Debug Console  Terminal  Ports

● PS C:\Users\saisu\Projects\dsa> gcc Q2.c -o Q2
  >>
● PS C:\Users\saisu\Projects\dsa> .\Q2
  BFS Traversal: 2 0 1 3
● PS C:\Users\saisu\Projects\dsa> █
```

Question 3: Depth First Search (DFS) Traversal

Problem Description:

Implement Depth-First Search (DFS) graph traversal recursively to explore as deep as possible along each branch before backtracking.

Sample Input: Start vertex: 2 | Sample Output: DFS Traversal: 2 0 1 3

Output Screenshot:



```
Problems  Output  Debug Console  Terminal  Ports

● PS C:\Users\saisu\Projects\dsa> gcc Q3.c -o Q3
  >>
● PS C:\Users\saisu\Projects\dsa> .\Q3
  DFS Traversal: 2 0 1 3
● PS C:\Users\saisu\Projects\dsa> █
```

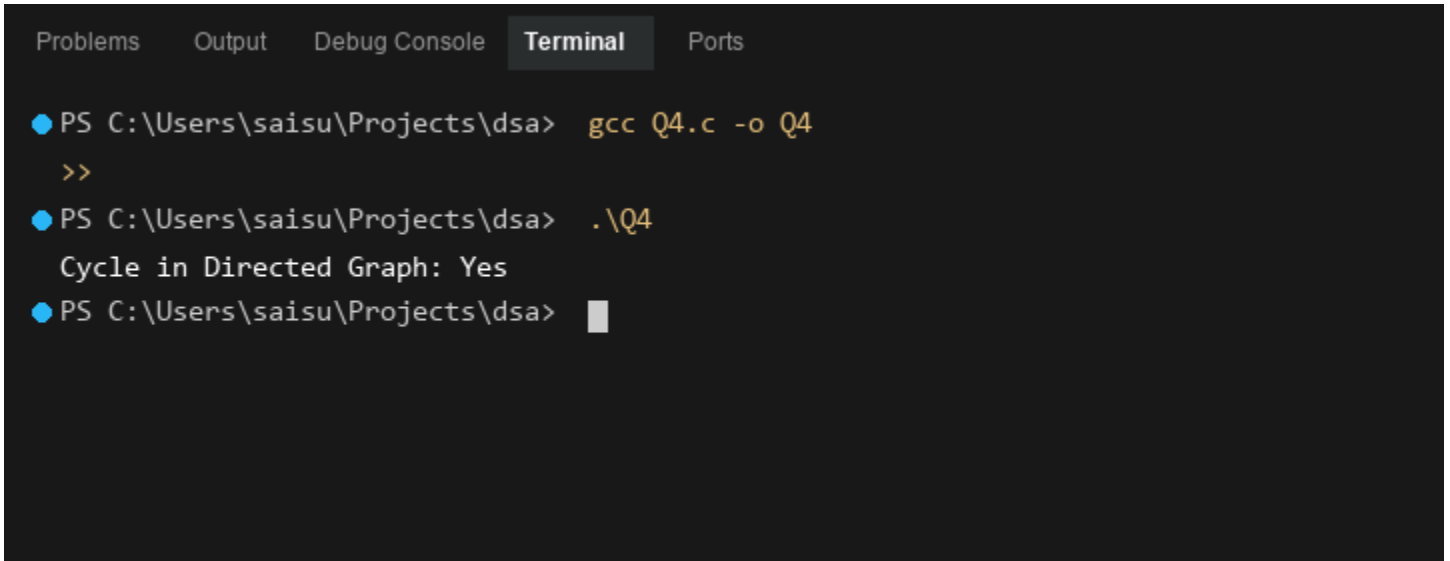
Question 4: Detect Cycle in Directed Graph

Problem Description:

Detect if a directed graph contains a cycle using DFS with a recursion stack tracking active back-edges.

Sample Input: Edges: 0->1, 1->2, 2->0 | Sample Output: Cycle Detected: Yes

Output Screenshot:



```
Problems  Output  Debug Console  Terminal  Ports

● PS C:\Users\saisu\Projects\dsa> gcc Q4.c -o Q4
  >>
● PS C:\Users\saisu\Projects\dsa> .\Q4
  Cycle in Directed Graph: Yes
● PS C:\Users\saisu\Projects\dsa> █
```

Question 5: Detect Cycle in Undirected Graph using Union-Find

Problem Description:

Detect cycles in an undirected graph using the Disjoint Set Union (DSU / Union-Find) algorithm with path compression.

Sample Input: Edges: (0,1), (1,2), (0,2) | Sample Output: Graph Contains Cycle: True

Output Screenshot:



```
Problems  Output  Debug Console  Terminal  Ports

● PS C:\Users\saisu\Projects\dsa> gcc Q5.c -o Q5
>>
● PS C:\Users\saisu\Projects\dsa> .\Q5
Graph Contains Cycle: True
● PS C:\Users\saisu\Projects\dsa> █
```

Question 6: Dijkstra's Shortest Path Algorithm

Problem Description:

Find the shortest paths from a single source vertex to all other vertices in a non-negative weighted graph in $O(V^2)$ or $O(E \log V)$ time.

Sample Input: Source = 0, 5 vertices | Sample Output: Distances: 0 4 12 19 21

Output Screenshot:



```
Problems  Output  Debug Console  Terminal  Ports

● PS C:\Users\saisu\Projects\dsa> gcc Q6.c -o Q6
>>
● PS C:\Users\saisu\Projects\dsa> .\Q6
Vertex  Distance from Source 0
0  0
1  4
2  12
3  19
4  28
● PS C:\Users\saisu\Projects\dsa> 
```

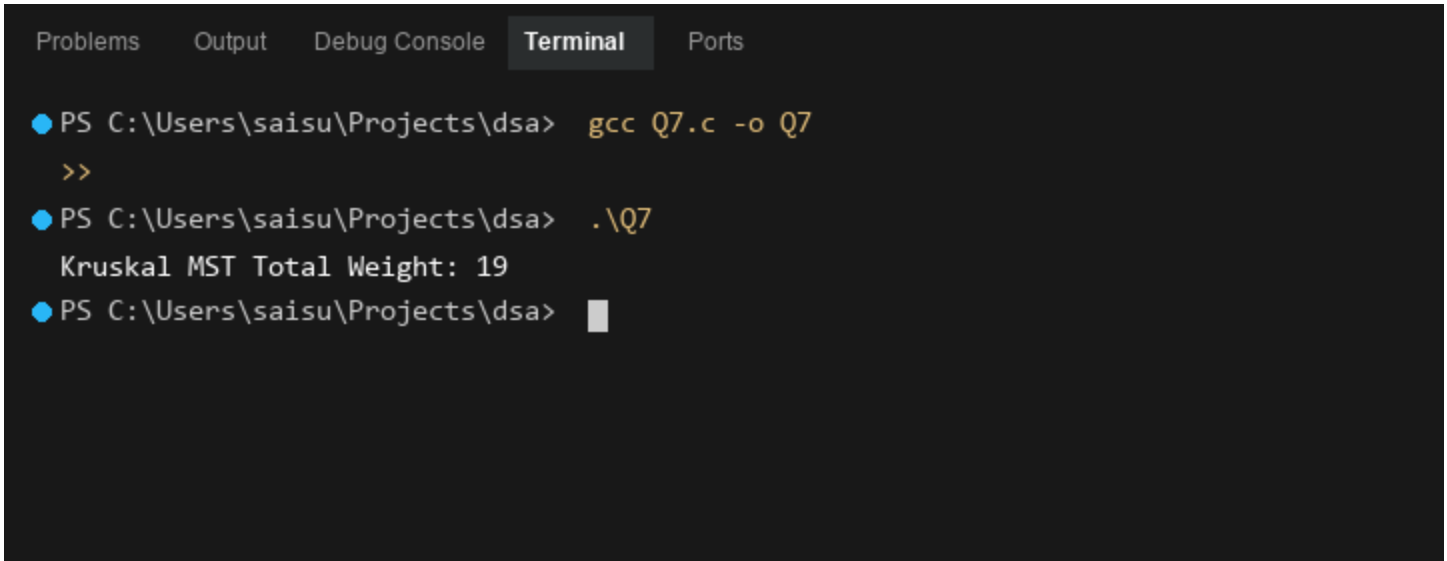
Question 7: Kruskal's Minimum Spanning Tree Algorithm

Problem Description:

Find the Minimum Spanning Tree (MST) of a weighted undirected graph by sorting edges in ascending weight and applying Union-Find.

Sample Input: 4 vertices, 5 edges | Sample Output: MST Total Weight: 19

Output Screenshot:



```
Problems  Output  Debug Console  Terminal  Ports

● PS C:\Users\saisu\Projects\dsa> gcc Q7.c -o Q7
  >>
● PS C:\Users\saisu\Projects\dsa> .\Q7
  Kruskal MST Total Weight: 19
● PS C:\Users\saisu\Projects\dsa> █
```

Question 8: Prim's Minimum Spanning Tree Algorithm

Problem Description:

Find the Minimum Spanning Tree (MST) of a connected weighted graph starting from vertex 0 using a greedy minimum cut key array.

Sample Input: 5 vertices weighted graph | Sample Output: Prim MST Total Weight: 16

Output Screenshot:



```
Problems  Output  Debug Console  Terminal  Ports

● PS C:\Users\saisu\Projects\dsa> gcc Q8.c -o Q8
  >>
● PS C:\Users\saisu\Projects\dsa> .\Q8
  Prim's MST Total Cost: 10
● PS C:\Users\saisu\Projects\dsa> █
```

Question 9: Topological Sort of DAG (Kahn's Algorithm)

Problem Description:

Produce a linear ordering of vertices for a Directed Acyclic Graph (DAG) using in-degrees and a queue (Kahn's Algorithm).

Sample Input: 6 vertices DAG | Sample Output: Topological Order: 4 5 0 2 3 1

Output Screenshot:



```
Problems  Output  Debug Console  Terminal  Ports

● PS C:\Users\saisu\Projects\dsa> gcc Q9.c -o Q9
>>
● PS C:\Users\saisu\Projects\dsa> .\Q9
Topological Sort: 4 5 0 2 3 1
● PS C:\Users\saisu\Projects\dsa> █
```

Question 10: Bellman-Ford Shortest Path Algorithm

Problem Description:

Find shortest paths from a single source to all vertices in a graph that may contain negative weight edges and detect negative cycles in $O(V \cdot E)$ time.

Sample Input: 5 vertices, 8 edges (some negative) | Sample Output: Shortest distances printed, No negative cycle

Output Screenshot:



```
Problems  Output  Debug Console  Terminal  Ports

● PS C:\Users\saisu\Projects\dsa> gcc Q10.c -o Q10
>>
● PS C:\Users\saisu\Projects\dsa> .\Q10
Vertex Distances from Source 0:
Vertex 0: 0
Vertex 1: -1
Vertex 2: 2
Vertex 3: -2
Vertex 4: 1
● PS C:\Users\saisu\Projects\dsa> █
```